Algoritmi paraleli si distribuiți

1. Introducere

Cristian Mihăescu

cristian.mihaescu@edu.ucv.ro

Cuprins

- Aspecte fundamentale
  - Introducere în calculul parallel si sistemele distribuite
  - Platforme pentru calculul paralel
  - Principii de proiectare ale algoritmilor paraleli
  - Modelarea programelor paralele
  - Programarea cu ajutorul trimiterii de mesaje
  - Programarea cu ajutorul memoriei partajate
- Algoritmi paraleli/distribuiți
  - Sortări, Căutări
  - Algoritmi pe matrici dense
  - Algoritmi pe grafuri, Programare dinamică
  - Algoritmul *bakery*, problema generalilor bizantini, map-reduce

Motivația

- Paralelismul/distributivitatea reprezintă un pas înainte in ceea ce priveste programarea calculatoarelor:
  - Trecere de la ***gândirea algoritmică secvențială*** *(en. Algorithmic thinking)* la ***gândirea paralelă*** (en. Parallel thinking).
- Multe calculatoare actuale au deja chip-uri multicore
- Utilizarea adecvată a calculatoarelor paralele este utilă în inginerie, știință, afaceri, etc.
- Provocare intelectuală mare în zona CS: modele, limbaje de programare, algoritmi, HW
- Multe probleme practice au solutii algoritmice paralelizabile. Acest fapt permite accelerarea!
- Exemple: Prognoza vremii, mișcarea astrelor, genetică.

Domenii de aplicație care necesită APD

- Sănătate
- Inginerie
- Cercetare spațială
- Planificare urbană
- Finanțe și afaceri: Bitcoin

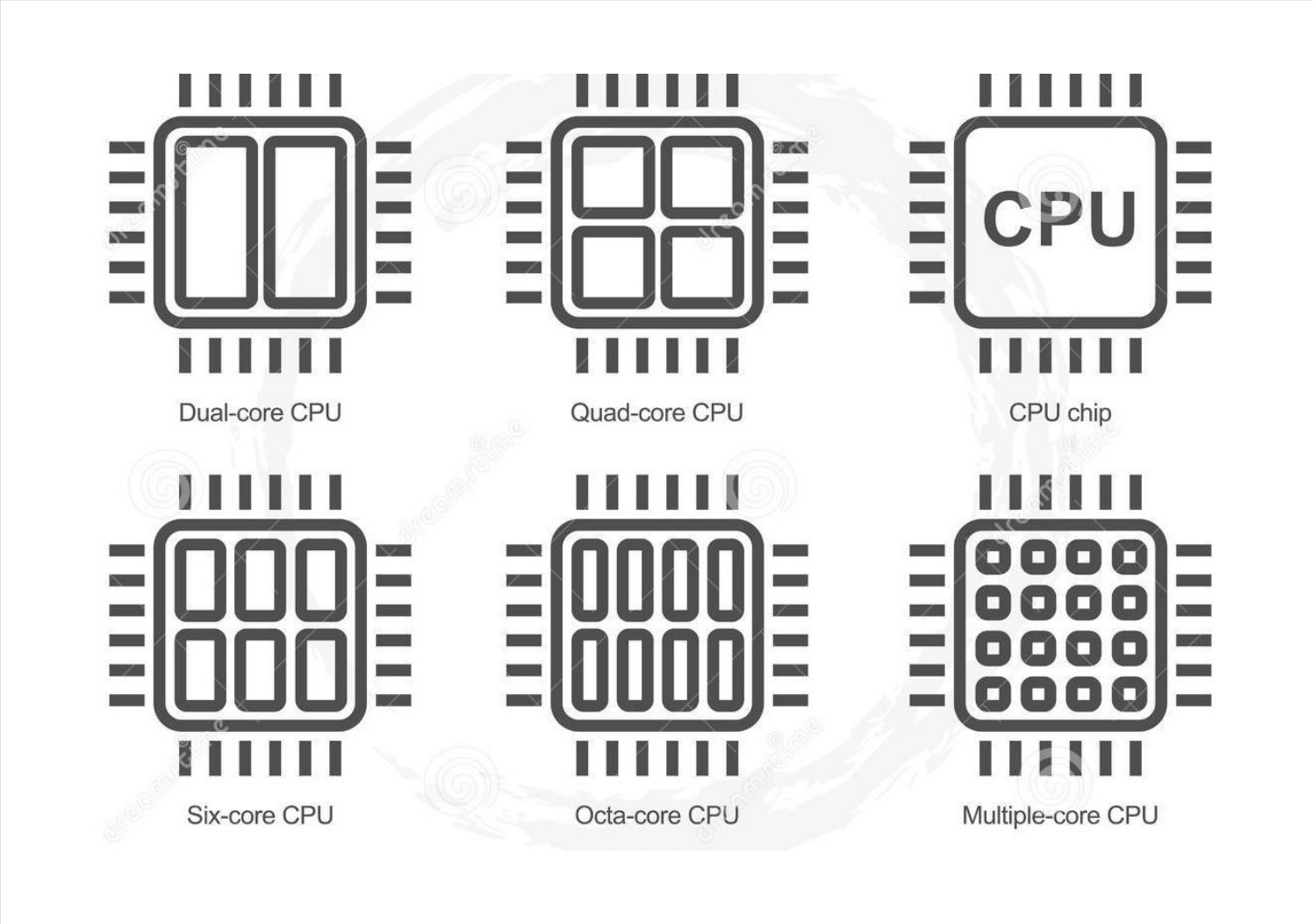
https://builtin.com/hardware/high-performance-computing-applications

Mae Rice, November 4, 2019, Updated: April 6, 2020

Tipuri de sisteme

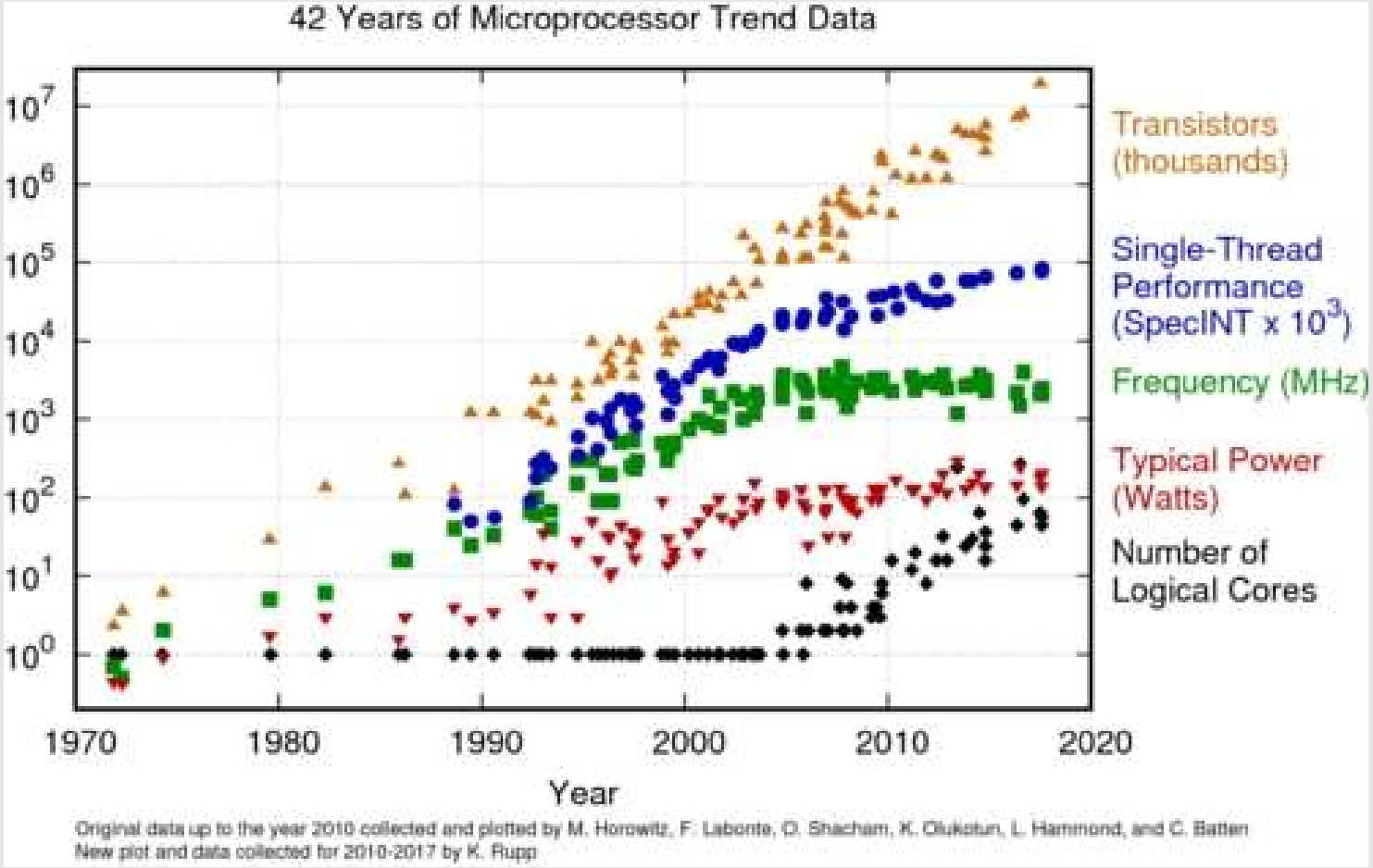
- Calculatoare personale
  - Nu sunt distribuite dar in prezent sunt multiprocesor
- Sisteme embedded
  - Rulează pe un singur procesor sau pe un grup integrat de procesoare
- Sisteme distribuite
  - Aplicația rulează pe un gup de sisteme cuplate/integrate slab (en., loosely coupled/integrated) care cooperează cu ajutorul unei rețele.

Multicore chip

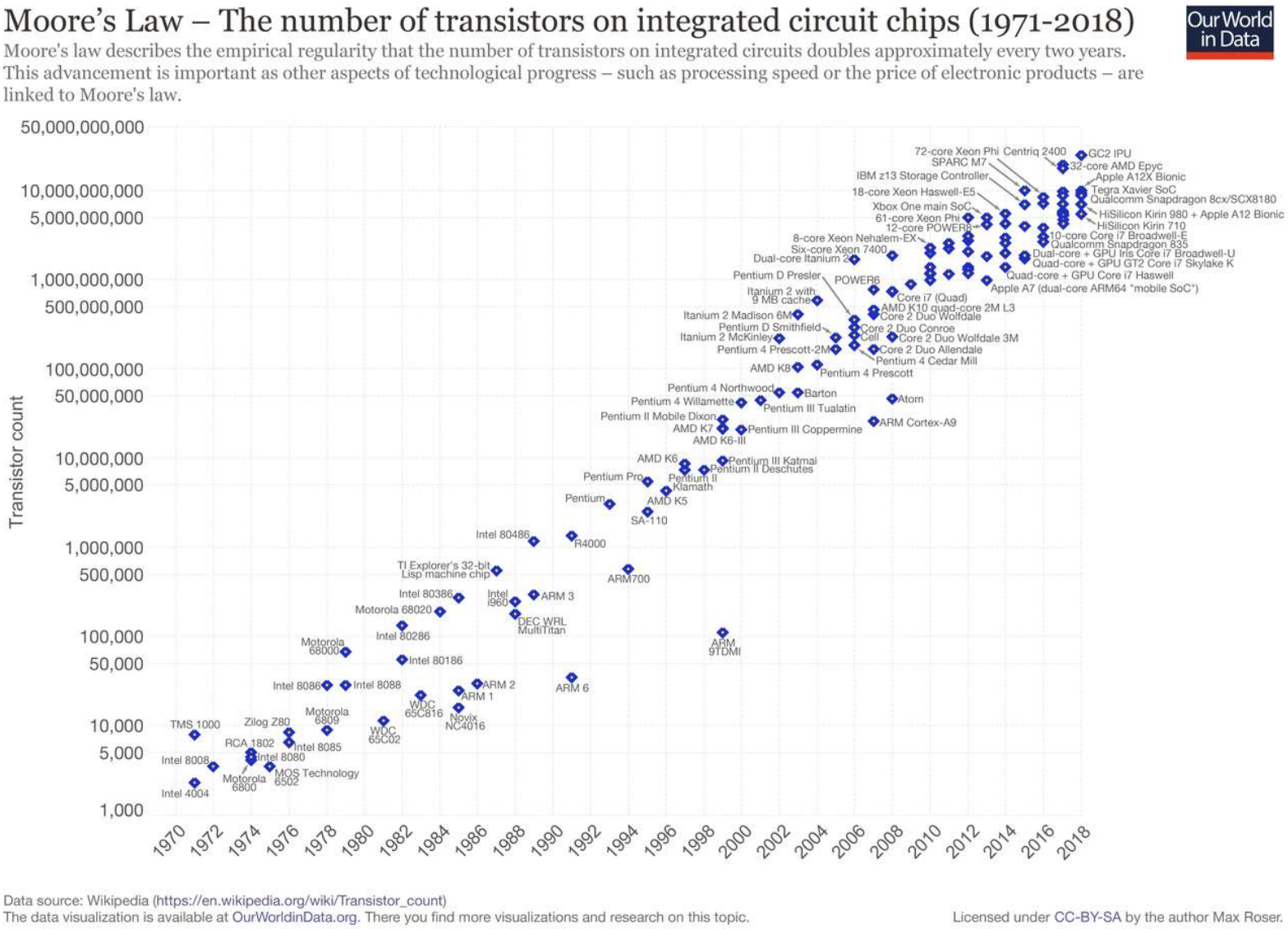


Facts: 55th Anniversary of Moore's Law

https://www.infoq.com/news/2020/04/Moores-law-55/



Moore's Law



Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

Licensed under CC-BY-SA by the author Max Roser.

https://en.wikipedia.org/wiki/Transistor_count

Elementele calculului paralel

- Hardware
  - Mai multe procesoare
  - Mai multe memorii
  - Retea de interconectare de mare viteză
- Sistem de operare
  - Sistem de operare care permite calcul paralel
  - Elemente de programare care permit exprimarea/orchestrarea concurenței
- Aplicații software
  - Biblioteci/sisteme care permit paralelizare/distribuție

SCOP: Utilizarea Hardware, Sistem de operare si Aplicațiilor software pentru:

1) obținerea **accelerării** (en., speedup)

2) rezolvarea de probleme care necesită o cantitate mare de memorie

Platforma de calcul paralel

- Organizarea logică
  - Modul în care utilizatorul vede mașina prin intermediul sistemului de operare
- Organizarea fizică
  - Arhitectura fizică a sistemului
- Arhitectura fizică este de obicei independentă de arhitectura logică

Scopul calculului paralel

- De obicei, arhitecturile convenționale sunt alcătuite dintr-un **procesor**, o **memorie**, și o **cale de acces** între memorie și procesor (en., datapath).
- Fiecare dintre aceste 3 componente poate deveni o gâtuire (en., bottleneck)
- Paralelismul adresează fiecare dintre aceste componente
- Fiecare aplicație poate integra aspecte diferite ale paralelismului:
  - Aplicațiile care procesează cantități mari de date
  - Aplicațiile server folosesc bandă largă
  - Aplicațiile științifice necesită putere mare de calcul si memorie

Context

- **Calculul paralel/distribuit** este executarea simultană logică a mai multor procese a căror execuție se poate întrețese (en., interleave) în moduri diferite și complexe.
- **Comunicarea si coordonarea între procese** se realizează prin transmiterea de mesaje (en., message passing) sau partajarea de memorie (en., shared-memory) și cu ajutorul conceptelor algoritmice ca atomicitate, consens, așteptare condiționată.
- **Obținerea accelerării în practică** necesită înțelegerea algorimilor paraleli, a strategiilor de descompunere, a arhitecturii sistemelor, a strategiilor de implementare, și a analizei performanțelor.

Context (cont.)

- **Sistemele distribuite** adresează problemele legate de securitate și toleranta la defecte (en., fault tolerance), necesitatea întreținerii de replici ale datelor și adresarea problemelor legate de rețeaua folosită.

- **Toleranta la defecte** se referă la abilitatea unui sistem (ex., calculator, rețea, cluster în cloud, etc.) de a-si continua rularea fără întrerupere atunci când una sau mai multe componente se defectează (en., fail).

Definiții / Termeni

- **Paralelism** = Utilizarea simultană de resurse de calcul (ex., procesoare, memorii) pentru a obține accelerarea unui program sau a unei aplicații.
- **Concurență** = Administrarea eficientă și corectă a accesului la resurse de calcul.
- **Activitate/Task** = calcul (en., computation) poate rula concurent cu alte activități. Exemple: un program, un proces, un fir de execuție, sau un echipament hardware dedicat.
- **Atomicitate** = Proprietatea unei activități de a fi indivizibilă, adică nu se putea impărți în activități care să poată rula în paralel. De exemplu: setarea biților într-un word, transmiterea unui singur pachet, finalizarea unei tranzacții.

Definiții / Termeni (cont.)

- **Aprobarea unanimă (en., Consensus)** = Acordul între două sau mai multe activități în ceea ce privește un predicat. De exemplu, valoarea unui numărător, proprietarul unui zăvor (en., lock), sau terminarea unui fir de execuție.
- **Consistență (en., Consistency:)** = Reguli si proprietăți convenite intre activități în ceea ce privește valorile variabilelor sau mesajele trimise. Această situație poartă denumirea de *data race*.

De exemplu, consistența secvențială stabilește că valorile tuturor variabilelor din memoria partajată a unui program paralel sunt aceleași cu ale unui singur program care realizează/permite accesul întrețesut la memorie pentru activitățile definite.

Definiții / Termeni (cont.)

- **Multicast** = Reprezintă transmiterea unui mesaj către mai mulți destinatari (en., recipients) fără nici o constrângere în ceea ce privește ordinea în care sunt recepționate mesajele. Un eveniment este un mesaj trimis către o mulțime stabilită de ascultători/abonați (en., listeners or subscribers). [Notă: Observer pattern]
- **Calculator paralel** = un calculator cu mai multe procesoare care pot lucra impreună pentru a rezolva o problemă de calcul.
  - *Implicatie*: Notiunea de paralelism nu exista in situatia sismelor cu un singur procesor.

  1) toate procesoarele partajeaza aceeași memorie;

  2) procesoarele comunică între ele accesând această memorie partajată

Definiții / Termeni (cont.)

- **Program paralel** = un program care se executa pe mai multe procesoare in acelasi timp.
- **Calculator distribuit** = mai multe calculatoare/sisteme independente conectate intr-o rețea
  - Apar pentru utilizatori/aplicații ca un singur sistem (ex., rețea de calculatoare)
  - Fiecare procesor are memoria lui proprie, ceea ce <u>impiedică</u> folosirea memoriei partajate pentru comunicare.
  - Procesoarele comunică prin transmiterea de mesaje.
  - Cu toate ca transmiterea de mesaje este mai înceată decât memoria partajată, transmiterea de mesaje se scalează mai bine pentru multe procesoare si este o solutie mai ieftină.

Avantajele sistemelor distribuite

- **Partajarea datelor**: permite multor utilizatori accesul la aceleași date în mod transparent.
- **Partajarea resurselor**: permite accesul mai multor utilizatori la aceleasi periferice (ex., imprimantă).
- **Comunicarea**: Permite comunicarea simultană (ex., email, chat, etc.)
- **Flexibilitate**: Permite impărțirea sarcinilor pe mai multe sisteme disponibile.

Avantajele sistemelor distribuite (cont.)

- **Prețul**: O colecție de sisteme/microprocesoare costă mai puțin decât un calculator mainframe (ex., Sun Enterprise 1000)
  - https://www.cnet.com/products/sun-enterprise-10000-ultrasparc-ii-400-mhz-monitor-none-series/ (~100K USD)
- **Viteza**: putere mai mare de calcul decât un mainframe prin distribuția încărcării (en. Load balancing).

Avantajele sistemelor distribuite (cont.)

- **Distributivitatea inerentă/nativă**: Unele aplicații sunt inerent/nativ distribuite (ex., lanțurile de magazine).

- **Siguranța/increderea în funcționare** (en., reliable): Chiar dacă o masină se defectează, intregul sistem va continua funcționarea datorită redundanței datelor si conexiunilor.

Limitări ale sistemelor distribuite

- **Software**: Dificultatea dezvoltării de aplicații pentru sistemele distribuite
- **Rețea**: Întârzieri, saturație, pierderea pachetelor
- **Securitate**: Acces facil la date.
- **Aprobarea unanimă** (en., consensus)
  - Două noduri/sisteme trebuie să convină asupra unei valori utilizând comunicarea prin mesaje prin canale ***nesigure*** (en., unreliable).
    - Nodurile propun o valoare, dar unele se defectează sau nu raspund
  - Algoritmul trebuie să se asigure că:
    - Toate nodurile funcționale decid; Toate nodurile au aceeași valoare; Se alege valoarea finală numai din valorile propuse.

Limitări ale sistemelor distribuite

- Transmiterea atomică (en., atomic broadcast)
  - Un nod transmite mesajul
  - Daca se primeste mesajul atunci un nod care funcționează corect retransmite inapoi mesajul
  - Toate nodurile care funcționeaza corect transmit același mesaj
  - Mesajele sunt transmise în aceeași ordine
- In situația transmirerii atomice
  - O putem folosi pentru a ajunge la consens (i.e., aprobare unanimă)
  - Fiecare nod transmite o propunere
  - Nodurile aleg prima propunere primită; mesajele sunt primite în aceeși ordine astfel încât toate nodurile vor lua aceeași decizie
  - Având consens îl putem folosi pentru a avea transmitere atomică

Concluzie: Transmiterea atomică este echivalentă cu consensul.

Exemplu: Problema celor 2 generali

https://en.wikipedia.org/wiki/Two_Generals%27_Problem

- Doi generali trebuie să coordoneze un atac
  - Trebuie să agreeze un moment al atacului; - Înving doar dacă atacă simultan
  - Comunică prin meageri; - Mesagerii pot fi uciși
- Solutie:
  - P1. G1 trimite momentul atacului lui G2. Cum știm că G2 primeste mesajul?
  - P2. G2 confirma primirea mesajului. Cum știm că G1 primeste confirmarea?
- Aceasta este prima problemă de comunicare între calculatoare care s-a demonstrat ca nu are soluție în situația in care apar eșecuri (en., failures) de comunicare în mod arbitrar.

1) Aceasta este situația în sistemele distribuite reale!
2) Problema generalilor Bizantini (generalizarea problemei celor 2 generali) arata cand un sistem distribuit poate continua să funcționeze în ciuda defectării unor componente.

Paralel sau distribuit?

Caracteristică	Paralel	Distribuit
Scop general	Viteză	comoditate
Interacțiuni	Frecvente	Rare
Granularitate	Mare	Mică
Siguranță în funcționare (en., reliable)	Presupus adevarată	Posibil neadevătară

Calcul paralel: Eficiența se reduce la optimizarea comunicației între procesoare, aceasta trebuie minimizată.

Calcul distribuit: Aspectul critic este dat de coordonarea între resurse (ex., sisteme, noduri). Apare necesitatea redundanței din cauza defectărilor (en. failures)!

Paralel/Distribuit

- **Sistem paralel** = Colecție de procesoare strâns conectate dintr-un singur sistem. Fiecare procesor execută într-o manieră semi-independentă o sarcina si se coordonează cu celelalte procesoare din timp în timp.

- **Sistem distribuit** = Colecție de sisteme autonome (care NU sunt strâns conectate) care comunică într-o rețea de cat mai mare viteză și care comunică pentru a îndeplini o sarcină.

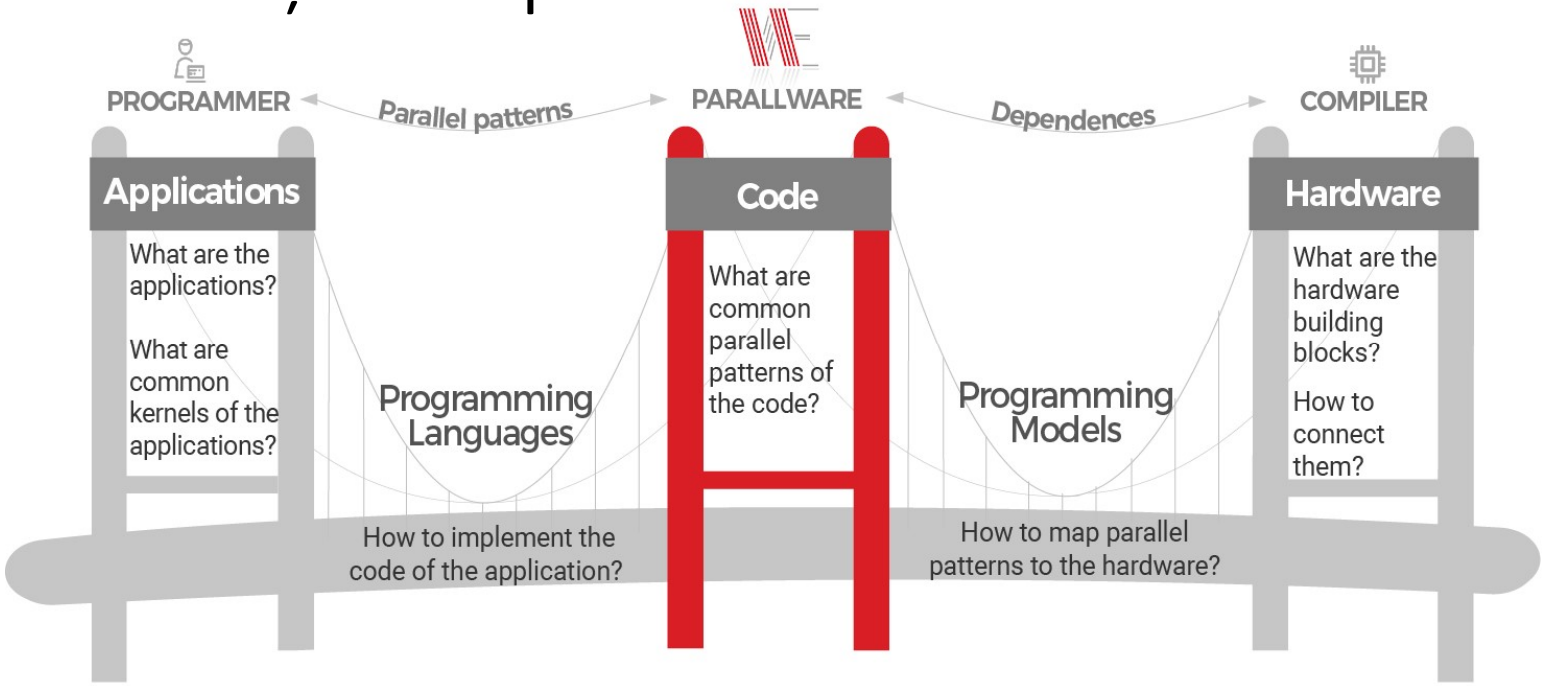
Leslie Lamport: *"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable "*

Obs: Procesarea paralelă în sistemele distribuite nu numai că este posibilă, dar este și o opțiune bună în ceea ce privește costurile.

Scopul procesarii paralele/distribuite

- Fie **T** timpul necesar rezolvarii unei probleme pe un singur procesor.
- Dacă avem la dispozitie ***p*** procesoare, am putea obține o accelerare care să permită rezolvarea problemei in **T/p** timp?
  - Este acest lucru posibil in practică?
  - Care sunt limitarile? Coordonarea, comunicarea, balansarea.
- Exemplu de problemă: Determinati toate elementele negative dintr-un tablou
    - Impartim tabloul in segmente in functie de numarul de procesoare.
    - Fiecare procesor determină numarul de elemente negative din segmentul alocat.
    - La final, adunăm toate valorile detrminate.

De la secvențial la paralel



Parallel Computing Laboratory at UC Berkeley: "*Writing programs that scale with increasing numbers of cores should be as easy as writing programs for sequential computers*"
https://www.appentra.com/parallware-method-assisted-parallelism-discover-parallel-patterns/

Dificultăți în procesarea paralelă

- Depanarea (en., debugging)
  - Dificultatea apare din cauza greutății cu care putem replica situatia apărută. Refacerea/simularea contextului în care a aparut o eroare este în multe situații o operatie dificil de realizat. Acest lucru se datorează intrețeserii (en., interleaving).
- Corectitudinea
  - Demonstrarea corectitudinii unui algoritm paralel este o provocare.
  - Incorectitudinea poate conduce la *undefined behaviour* iar depanarea este de obicei dificilă.
- Benchmark
  - Evaluarea performațelor (i.e., accelerarea) prin rulari/simulari este aspectul principal. La programarea secventuală (mai ales pe input-uri mici) acest aspect este de multe ori (incorect!) ignorat.

Dificultăți în procesarea paralelă (cont.)

- Comunicarea între procesoare
- Dificultatea abordarii paralele a algoritmului/procesării și ulterior dezvoltarea, testarea si implementarea soluției.
- Dintre soluții:
  - Proiectarea optimă și integrată a procesoarelor/memoriilor.
  - Apariția și dezvoltarea microchip-urilor multiprocesor.
  - Apariția de modele standard de programare paralela si comunicare in diverse limbaje de programare la diverse nivele:
    - Limbaje de programare: C/C++, Java, C#, etc.
    - Sisteme de programare: OpenMP, MPI
    - Sisteme pentru GPU: CUDA, OpenCL, OpenACC
  - Folosirea HW comun si standardizarea SW
  - NU este o buna practică:
    - Utilizarea sistemelor paralele pe aplicații secvențiale.
    - Paralelizarea aplicațiilor fară să avem sisteme paralele la dispoziție.

Contributori

- **Dijkstra, Edsger W.** (*1972 ACM Turing Award*) and C.S. Scholten, Predicate Calculus and Program Semantics, Texts and Monographs in Computer Science. Springer-Verlag, 1989.
  - Susține "calculational proof style" pentru argumentarea matematică. A utilizat metoda propusă în geometrie, algebră liniară, teoria grafurilor, proiectarea programelor secvențiale si distribuite.

  https://amturing.acm.org/award_winners/dijkstra_1053701.cfm
- **Hoare, Antony C.** (*1980 ACM Turing Award*)

- Propune limbajul CSP în care interacțiunea dintre programe era limitată de comunicații prestabilite. CSP devine ulterior aplicație si apoi transputer (https://en.wikipedia.org/wiki/Transputer).

- Transputer = serie de microprocesoare din anii 1980 care are memorie si comunicare serială proiectată în vederea calculului paralel.

https://amturing.acm.org/award_winners/hoare_4622167.cfm

Contributori

- **Dennis M. Ritchie** (*1983 ACM Turing Award*) : A proiectat sistemul de operare distribuit *Inferno* și limbajul de programare *Limbo* în 1995. Inferno a fost proiectat pentru cutiile de cabluri ale televizoarelor (en., television set-top boxes) și pentru telefoanele avansate.

https://amturing.acm.org/award_winners/ritchie_1506389.cfm

- **Fernando Corbato** (*1983 ACM Turing Award*): ” For his pioneering work organizing the concepts and leading the development of the general-purpose, large-scale, time-sharing and resource-sharing computer systems, CTSS and Multics.”

https://amturing.acm.org/award_winners/corbato_1009471.cfm

Contributori

- **Milner Robin Arthur** (*1991 ACM Turing Award*):

CCS: Teoria generală a concurenței

Milner, R., A Calculus for Communicating Systems, Lecture Notes in Computer Science, Vol. 92, Springer, 1980.

https://amturing.acm.org/award_winners/milner_1569367.cfm

- **Butler Lampson** (*1992 ACM Turing Award*):

”For contributions to the development of distributed, personal computing environments and the technology for their implementation: workstations, networks, operating systems, programming systems, displays, security and document publishing.”

https://amturing.acm.org/award_winners/lampson_1142421.cfm

Contributori

- **Vinton G. Cerf** și **Robert E. Kahn** (*2004 ACM Turing Award*):

”led the design and implementation of the Transmission Control Protocol and Internet Protocol (TCP/IP) that are the basis for the current internet. ”

In 1983, Kahn a propus ”ARPA’s Strategic Computing Initiative” unprogram de cercetare de 1mld USD care includea proiectarea de chip-uri, arhitecturi de calcul paralel, si inteligență artificială.

https://amturing.acm.org/award_winners/cerf_1083211.cfm

Contributori

- **Barbara Liskov** (*2008 ACM Turing Award*):

” For contributions to practical and theoretical foundations of programming language and system design, especially related to data abstraction, fault tolerance, and distributed computing. ”

https://amturing.acm.org/award_winners/liskov_1108679.cfm

- **Leslie Vailant** (*2010 ACM Turing Award*):

” For transformative contributions to the theory of computation, including the theory of probably approximately correct (PAC) learning, the complexity of enumeration and of algebraic computation, and the theory of parallel and distributed computing.”

https://amturing.acm.org/award_winners/valiant_2612174.cfm

Contributori

- **Leslie Lamport** (*2013 ACM Turing Award*):

” For fundamental contributions to the theory and practice of distributed and concurrent systems, notably the invention of concepts such as causality and logical clocks, safety and liveness, replicated state machines, and sequential consistency. ”

https://amturing.acm.org/award_winners/lamport_1205376.cfm

Contribuții:

- Algoritmul Bakery
- Bazele programării concurente: "loop freedom”, consistența secvențială, registri atomici.
- Bazele sistemelor distribuite: ceasuri logice, înțelegerea Bizantină (en., Byzantine Agreement), Replicarea masinilor cu stări (State Machine Replication)
- Verificarea și specificarea formală a programelor
- LaTeX

Contributori [alte domenii]

- 2014: Michael Stonebraker: ”... ***modern <u>database systems</u>***”
- 2015: Diffie Whitfield: ” ***asymmetric public-key <u>cryptography</u>, ... digital signatures, and a practical cryptographic key-exchange method***”
- 2016: Barnes-Lee-Tim: ”***For inventing the <u>World Wide Web</u>***”
- 2018: Bengio, Hinton, LeCun: ”***<u>deep neural networks</u> a critical component of computing***”
- 2019: Catmull, Hanrahan: ” ***<u>3D computer graphics</u>, and the impact of computer-generated imagery (CGI) in filmmaking***”

Cine si ce?

- https://top500.org/ oferă statistici despre sistemele paralele existente: producători, utilizatori, locații, aplicații, etc.

Evaluare

- ***Laborator***: **40%** [minim nota 5]
  - MPI: http://software.ucv.ro/~cpoteras/pda/index.html
  - Paralelism in C++ : Threads / STL Paralel
  - Ștefan Popescu (4sg.) + Renato Ivănescu (2sg.)
- ***Proiect***: **30%** : Rezolvă o problema folosind:

1) Limbaje de programare:

- POSIX Threads, Pthreads library for C ; C++ : Java-Style Synchronization ; C++ Threads, Parallel STL C++ algorithms; Java: Threads, synchronized methods, external locks, java.util.concurrent; python parallel computing; C# parallel computing

2) Sisteme/Frameworks de programare (en., programming system)

-OpenMP; MS-MPI (https://docs.microsoft.com/en-us/message-passing-interface/microsoft-mpi) ; MPI-3, AdaptiveMPI, Charm++ ; MPJ Express (http://mpj-express.org); PVM.

3) Sisteme/Frameworks de programare cu GPU

- CUDA/OpenACC/OpenCL

4) Alte tehnologii/unelte: Blockchain, Hadoop.

- ***Examen oral***: **30%** [minim nota 5]

- exmen partial la mijlocul semestrului

Evaluare: Proiect [30%]

- S1->S3 [2 sapt.] – Stabilirea temei: enunț, limbaje de programare, frameworks utilizate.
  - Algoritm + limbaj/abordări
- S3->S12 [10 sapt.]: Dezvoltarea soluțiilor
  - Cerințe minime:
    - soluție secvențială
    - 2-3 abordări paralele: limbaje de programare, sisteme, etc.
    - Comparatii ale accelerării în funție de dimensiunea intrării, limbaj, sistem, etc.
    - Minim 3 prezentări intermediare (S6, S8, S10)
- S12->S14[2 sapt.]: Evaluarea proiectelor
  - În cadrul laboratorului/cursului

Proiect: Exemple de probleme si abordări

- Probleme pe grafuri:
  - Traversarea grafurilor: BFS
  - Drumuri minime: Dijkstra, Roy-Floys, Delta-stepping
  - Arbori partiali de cost minim: Kruskal, Prim, Boruvka
  - Componente tari conexe: Kosaraju
  - Etc..
- Prelucrarea paralelă grafurilor pe CPU
  - C/C++, python, java, C# cu fire de execuție si/sau folosind biblioteci care implementeaza paralelism implicit sau explicit: C++ STL parallel, Java Tasks, C# Task Parallel Library (TPL) sau PLINQ, etc.
- Prelucrarea paralelă grafurilor pe GPU
  - https://www.dcs.warwick.ac.uk/~liganghe/papers/ACM-Computing-Surveys-2017.pdf
- https://neo4j.com/ (graph database management system)
  - https://neo4j.com/docs/graph-data-science/current/algorithms/dijkstra-single-source/

Referințe

- Computer Science Curricula 2013, Parallel and Distributed Computing (PD), https://www.acm.org/binaries/content/assets/education/cs2013_web_final.pdf
- Curriculum Initiative on Parallel and Distributed Computing - Core Topics for Undergraduates, http://tcpp.cs.gsu.edu/curriculum/
- Lynch, N. A. (1996). Distributed algorithms. Elsevier.
- Grama, Ananth, Vipin Kumar, Anshul Gupta, and George Karypis. Introduction to parallel computing. Pearson Education, 2003.

https://www-users.cs.umn.edu/~karypis/parbook/

- Designing and Building Parallel Programs, by Ian Foster
  https://www.mcs.anl.gov/~itf/dbpp/
- Livermore Computing Center Training Materials
  - https://hpc.llnl.gov/training/tutorials